

Unit 5 – Maintenance

Software Project Management

Learning Outcomes

- Identify various project management activities such as **project planning, monitoring, and control**.
- Discuss estimation of various parameters such as **cost, Efforts, and schedule/duration**.
- Determine **software metrics** using **Halstead's** software science.
- Understand the **COCOMO** for **software metrics**.

Software Project

- **Software Project** is the complete task of software development from requirements collection to software testing and maintenance carried in a **specified time**.
- **Every project has a target or goal with a start and end time**.
- **Project needs resources such as time, finance, knowledge, man power, and schedule**.

Software Project Management (SPM)

- **SPM** is an important part of software engineering from the stage of a **conceptual model** to the **developed software** stage.
- **SPM includes planning, scheduling, allocating resources, execution and delivery of software projects**.
- **SPM is under the constraints of budget and schedule planned**.

SPM

- The **goals** of the **SPM** are:
- **Maintain** the agreed **timing** of the **software delivery**.
- **Keep** the **project activities** under the **allotted budget**.
- **Fulfill** the client's **specifications**.
- **Make sure** that the **development team** is functioning well in a **happy atmosphere**.

SPM

- **Project management** in software engineering is **different** than other engineering practices.
- In software engineering the **product** is **intangible**.
- A **review** of the **progress** with **team members** is only possible in **software development**.
- When the **projects** are **large** the **project manager's** experience **may not be sufficient** to anticipate the **problems** in the **software development**.

SPM

- The **software processes** are **dynamic**, unlike electrical or **civil engineering** project tasks.
- When the software to be developed is a **part of a large engineering project**, it becomes **impossible to determine** whether there will be any software **development problems**.
- **SPM** is managed by the **project manager**.
- For any project, whether it is **small or big**, the **project manager** is employed for its completion.

Project Manager

- The **duties** of the **Project Manager** are:
- **Project planning**
- **Reporting**
- **Risk management**
- **People management**
- **Proposal writing**

Unit 5 – Maintenance

Software Project Management Activities

Software Project Management (SPM) Activities

- There are two **main activities** in **SPM** as follows.
- Project planning
- Project Monitoring and Control

Project Planning

- In this stage, the **project manager** is responsible for the **planning of the project** and **estimating** and **developing** the project schedule.
- The **different task allocation** to team members is done by the **project manager**.
- The manager will **monitor** the **progress of the project** development within the given **schedule** and **budget**.
- **Project planning** is completed before the **software development activity** begins.

Project Monitoring and Project Control

- The **project monitoring** and **control** activities ensure that the development proceeds as per **plan**.
- This activity also needs **changing the plan** based on the **situation**.
- Project managers report on the **progress** of a project, **risk**, and **people** management.

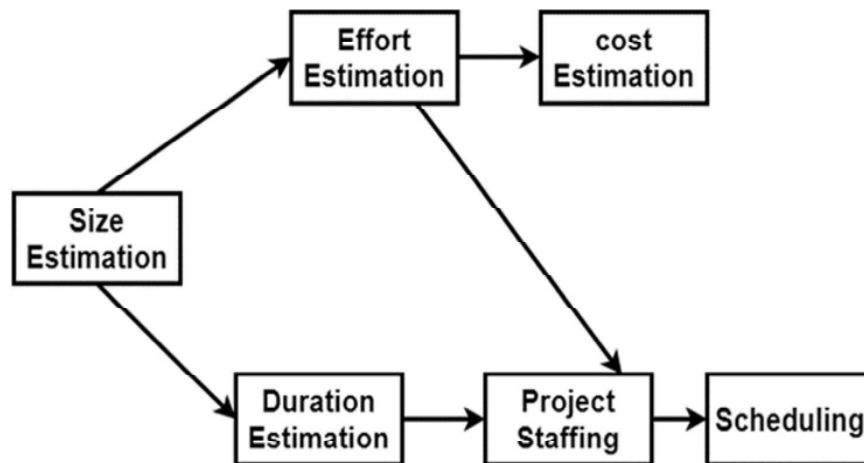
Project Monitoring and Project Control

- A software project can be with **several risks**. Risk management includes **identifying significant risks**, **classifying** them, and **finding the solution**.
- People management includes **issues of people** in **software development**, **different management styles**, and **ways** to **organize software development** teams.

Unit 5 – Maintenance

Software Project Planning

Project Planning



Project Planning

- Estimation of **project size**, **cost duration**, and **efforts**.
- The **size** of the **project** determines the **complexity** of the **project** in terms of **efforts** and **time** required for **software** development.
- The **cost** of the project refers to how much **finance** is necessary to **complete** a project.

Project Planning

- The **duration** refers to the **time** required to **complete project** development.
- The **efforts** refer to the **manpower** and other **resources** required for project completion.

Project Planning

- Preparing a schedule of **manpower** and **resources** such as **web**, **database**, **hardware**, etc.
- Organization and appointment of staff: In this activity, **analysis** is done to find out if the **existing staff** is sufficient for the **project** or if **new hiring** is required.
- **Configuration management** and **quality assurance** planning.
- The planning is outlined in **Software Project Management Plan (SPMP)** document.

SPMP Document

Name	Details
Introduction	Objectives, Functions, Performance issues, Constraints
Project estimates	Historical data used, Estimation techniques details, Cost, duration, effort estimates
Project Schedule	Work breakdown using Gantt and PERT chart
Project resource	Manpower, Hardware, and Software, skilled professionals

PERT (A Project Evaluation and Review Technique) and Gantt (named after the researcher who invented) charts are project management tools that enable project managers to track tasks and project status.

SPMP Document

Name	Details
Staff organization	Team formation and structure, Management reporting
Risk Management	Risk analysis, Risk identification, and Risk abatement methods
Project tracking and control plan	Process of tracking and reviewing the project's performance.
Miscellaneous activities	Software process, testing, verification and validation plan, testing, and configuration plan

Unit 5 – Maintenance

Project Size Estimation: LOC

Project Size Estimation

- The **cost, schedule, and time** of the **project** can be determined using the **size** of the **project**.
- This is the reason the **size of the project** must be determined very accurately.
- There are **two important metrics** commonly used in project size estimation:
 - **Lines of Code (LOC)**
 - **Function Point (FP)**

LOC

- This is the **simplest metric** used where the **project size** is calculated by **counting the number of source instructions** in the **program**.
- The **comments** and the **header file statements** are **excluded** from the count.
- For estimation of the **project size** at the **initial stages** of development, the **project size** can be divided into **modules and submodules**, and so on.

LOC

```
1. void Sort(int x[ ], int n) {  
2. //Sorting an array in ascending order  
3. int i, j, min, temp;  
4. for (i = 0; i < n - 1; i++) {  
5.     min = i;  
6.     for (j = i + 1; j < n; j++)  
7.         if (x[j] < x[min])  
8.             min = j;  
9.     temp = x[i];  
10.    x[i] = x[min];  
11.    x[min] = temp;  
12. }  
13. }
```

13 Lines
LOC = 12
Comment is ignored

LOC

- To estimate the **accurate LOC**, the **experience** in the **past** and the **skill** of the **software developer** are considered. **Programming skills** can be divided into **3 types** as shown below.

Skill Level	Developed Code
Efficient	Optimistic (S_{opt})
Average	Most Likely (S_m)
Poor	Pessimistic (S_{pess})

LOC

Skill Level	Developed Code
Efficient	Optimistic (S_{opt})
Average	Most Likely (S_m)
Poor	Pessimistic (S_{pess})

- **LOC** can be calculated using the formula:

- $$S = \left(\frac{S_{opt} + 4S_m + S_{pess}}{6} \right)$$

Example

- Example: Consider a software system, at the initial stage it is estimated that there are **5200 Sopt**, **7000 Sm**, and **9000 Spers**.
- Find the following for this application.
- The expected LOC of software
- $S = (Sopt + 4Sm + Spers) / 6$
- $= (5200 + 4 * 7000 + 9000) / 6$
- $= 7033 \text{ LOC}$

Example

- Consider a team of **two members** for software development.
- Determine the productivity rate.
- The productivity rate is given as **size/effort**.
- 2 men year = 24 months
- **7033 LOC -> 24 months**
- $7033 / 24 \rightarrow 1 \text{ month}$
- $= 293 \text{ LOC/man/month}$

LOC

- If the **duration** for the **software development** is estimated as **6 months**, then what is the **average manpower** with 2 members?
- **Manpower = Effort / Duration**
- **24/6 = 4 months**

LOC

- If **software development cost** is estimated as **Rs 2500 / man-month**, with a team of 2 members then
- what is the development cost of the project?
- **Solution:**
- **Development Cost = Development Effort * Cost/man-month**
- **= 24 * 2500**
- **= 60000**

LOC

- LOC is **easy to estimate** as compared to other techniques. It is a **commonly used cost estimation** method.
- However, it does **not consider the complexity** of the program such as the number of input/output, files, interfaces, functions, etc. It is **less efficient** due to dependency on the programming language.

Unit 5 – Maintenance

Function Point

Function Point

- **Function point (FP)** is used to measure the **size of software** based on the **functional view**.
- **FP** calculation considers the **number of inputs, outputs, queries, internal files, and external files**.
- **FP** determines the approximate development cost at the early stage. The **functionality** of the system is from the **user's point** of view.

Function Point

- **FP** was proposed in **1983** and it **overcomes many pitfalls** of the **LOC** metric.
- **FP** can be used to **estimate size** right from the **stage of problem specification**, unlike **LOC**.
- The **size of the software** depends on the **number of functions or features** included in the **project**.

Function Point

- The **more** the **number of functions** or **tasks** in the **project**, the **higher the size** of the project.
- **Other parameters** to estimate the **size** are the **number of files** and **interfaces** to the **project**, and **input/output data** values.

Function Point

- **Important Notations**
- **FP** = Function Point
- **UFP** = Unadjusted Function Point
- **VAF** = Value Adjusted Factor
- **DI** = Degree of Influence
- **TDI** = Total Degree of Influence

Function Point

- Compute the **Unadjusted Function Points (UFP)**
- Find **Total Degree of Influence (TDI)**
- Compute **Value Adjustment Factor (VAF)**
- Find the **Function Point Count (FPC)**

- $UFP = (EI*AWF) + (EO*AWF) + (EQ*AWF) + (ILF*AWF) + (EIF*AWF)$
- $TDI = 14*DI$ (there are 14 factors for each function)
- $VAF = 0.65 + (0.01*TDI)$
- $FP = UFP * VAF$

Function Point : 14 factors for TDI

Rate each factor on a scale of 0 to 5.



Number of factors considered (F_i)

1. Does the system require reliable backup and recovery ?
2. Is data communication required ?
3. Are there distributed processing functions ?
4. Is performance critical ?
5. Will the system run in an existing heavily utilized operational environment ?
6. Does the system require on line data entry ?
7. Does the on line data entry require the input transaction to be built over multiple screens or operations ?
8. Are the master files updated on line ?
9. Is the inputs, outputs, files, or inquiries complex ?
10. Is the internal processing complex ?
11. Is the code designed to be reusable ?
12. Are conversion and installation included in the design ?
13. Is the system designed for multiple installations in different organizations ?
14. Is the application designed to facilitate change and ease of use by the user ?

Function Point : 14 factors for TDI

1. reliable backup and recovery required?
2. data communication required?
3. are there distributed processing functions?
4. is performance-critical?
5. will the system run in an existing heavily utilized operational environment?
6. online data entry required ?
7. does the online data entry require the input transaction to be built over multiple screens or operations ?

Function Point : 14 factors for TDI

8. are the master files updated online?
9. is the inputs, outputs, files, or inquiries complex?
10. is the internal processing complex?
11. is the code designed to be reusable?
12. are the conversion and installation included in the design?
13. is the system designed for multiple installations in different organizations?
14. is the application designed to facilitate change and ease of use by the user?

Degree of Influence

Degree of Influence (DI)	Value
No Influence	0
Incidental	1
Moderate	2
Average	3
Significant	4
Essential	5

Calculation of UAF

Weighting Factor Table (generally average calculation is used)

Function Unit	LOW	AVG	HIGH
Number of external Inputs (EI)	3	4	6
Number of external outputs (EO)	4	5	7
Number of external inquiries (EQ)	3	4	6
Number of Internal Files (IF)	7	10	15
Number of external and internal interfaces (EIF)	5	7	10

Example

- Consider a software project with the following function units.
- 1) Number of user inputs = 20
- 2) Number of user outputs = 40
- 3) Number of inquiries = 5
- 4) Number of files = 4
- 5) Number of external interfaces = 2
- 6) Degree of influences is given as = 3,1,1,3,2,5,4,3,3,3,1,2,5,4
- 7) Weight Factors are given as = Average for which the values are 4,5,4,10,7
- Calculate the function point estimation of the project.
- Answer:
 - $FP = UFP * VAF$
 - $UFP = (EI*AWF) + (EO*AWF) + (EQ*AWF) + (ILF*AWF) + (EIF*AWF)$
 - $NAF = 0.65 + (0.01 * TDI)$
 - $TDI = 14 * DI$
-

Example

UFP Calculation			
Measurement Factor	Count	Weight Factor	Total
Number of external Inputs (EI)	20	4	80
Number of external outputs (EO)	40	5	200
Number of external inquiries (EQ)	4	4	16
Number of Internal Files (IF)	5	10	50
Number of external and internal interfaces (EIF)	2	7	14
Total UFP			360

Weighting Factor Table (generally average calculation is used)			
Function Unit	LOW	AVG	HIGH
Number of external Inputs (EI)	3	4	6
Number of external outputs (EO)	4	5	7
Number of external inquiries (EQ)	3	4	6
Number of Internal Files (IF)	7	10	15
Number of external and internal interfaces (EIF)	5	7	10

Calculation of DI = Average of given complex factor = average of 3,1,1,3,2,5,4,3,3,3,1,2,5,4 as given in the problem

$$DI = (3+1+1+3+2+5+4+3+3+3+1+2+5+4)/14 = 40/14 = 2.857$$

$$\text{Calculate } TDI = 14 * DI = 39.998$$

Calculate VAF

$$VAF = 0.65 + (0.01 * 39.998) = 1.0499 = 1.05$$

$$FP = UFP * VAF$$

$FP = 360 * 1.05$ (The value of UFP is taken as the average value from Table 1)

$$FP = 378$$

Function Point Summary

- Functionality is measured based on the **user's request and response**.
- The cost estimation of the software development and maintenance is **independent of the technology**.
- **Requirements can be verified** using the count of functions given by FP.
- The drawback of using FP is it **uses much judgment** which **may not be accurate in some cases**.
- There is **less research on FP** as compared to LOC.
- It is a **time-consuming** process.

Unit 5 – Maintenance

Function Point – Advantages and Drawbacks

Function Point- Calculation

- Consider a software project with numbers(weights) of input, output, inquiries, files, and external interfaces as –
- **20(4), 25(5),10(4),8(10),5(7) respectively.**
- The complexity adjustment of the project on average is **03.**
- What are the **function points** required for a software project?

Function Point- Calculation

- $FP = UFP * VAF$
- $UFP = \text{Count Value} = [(20*4) + (25*5) + (10*4) + (8*10) + (5*7)]$
- $= 360$
- $DI = (3+3+3+3+3+3+3+3+3+3+3+3+3+3)/14 = 42/14 = 3$
- $TDI = 14 * 3 = 42$
- $VAF = 0.65 + 0.01 * 42 = 1.07$
- OR We can directly write this equation as:
- $VAF = 0.65 + 0.01*(3+3+3...+3..14\text{times})$
- $= 0.65 + 0.01*(14*3)$
- $= 1.07$
- $FP = UFP * VAF = 360*1.07 = 385.2$

Function Point- Advantages

- FP can be used at the initial software development stage, unlike LOC.
- It does not depend on the programming language.
- Using FP, the productivity of the software can be determined using just the problem definition.
- With the FP method, the analysis of the same problem using different languages can be done and the best one can be determined.
- It can be used where there is a large amount of interface such as GUI.

Function Point- Drawbacks

- FP metric depends on subjective evaluations with a lot of judgment.
- This model is not suitable for real-time system software development.
- In many applications, it is necessary to convert FP to LOC for measuring software metrics.

Unit 5 – Maintenance

Project Estimation Techniques

Project Estimation Techniques

- The most fundamental parameters such as the **size of the project, efforts, duration**, and **cost** are determined in the project estimation technique.
- These parameters can be determined using **three** main categories of estimation techniques:
 - **Empirical estimation** techniques
 - **Heuristic** techniques
 - **Analytical estimation** techniques

Empirical estimation techniques

- An **educated guess** of project parameters is used.
- This guess is done using **common sense** and **experience** in the field.
- **Commonly** used **empirical project** estimation techniques are:
- **Expert judgment technique**: In this method, experts estimate the cost of different components of the software. The components refer to the modules and subsystems of the software. The drawback of this method is it may include human error or bias.

Empirical estimation techniques

- **Delphi estimation techniques**: In this method, a **coordinator provides the SRS document** to more than one estimator.
- Each **estimator uses several iterations** to **produce the estimate**.
- The coordinator distributes the **estimate responses** amongst estimators.
- After many rounds, the coordinator **compiles** and **prepares the final estimate**.
- This technique **avoids personal errors** or **bias** due to any considerations.

Heuristic Techniques - 1

- **Project parameters** are modeled using **suitable mathematical expressions**. This is a single variable model.

$$\text{Estimated Parameter} = c_1 * e_1^d$$

- **e** = Characteristic of the software estimated known as the **independent variable**. Estimated Parameters include **staff size, efforts, project duration**, etc.
- **c1** and **d1** are constants. The values of **c1** and **c2** are determined using the values from the previous projects.
- This is done using a **constructive cost estimation model (COCOMO)**.

Heuristic Techniques - 2

- Another project estimation model is the **multivariable cost estimation model**.
- The **multivariable model** is more accurate than a **single variable** as it includes **more amount of historical data**. The mathematical equation in this model is given as:

$$\text{Estimated Resource} = c_1 * e_1^d + c_2 * e_2^d + \dots$$

- **e1, e2, ...:** Independent characteristics of the **software** already submitted.
- **c1, c2, d1, d2, ...:** constants using historical data.

Unit 5 – Maintenance

COCOMO - Introduction

COCOMO

- **COCOMO** model is proposed by Barry Boehm in 1981 after considering a study of 63 projects.
- COCOMO is based on **LOC**.
- The different parameters such as **size, effort, cost, time**, and **quality** are estimated in this model.
- According to **Boehm**, the software projects can be divided as:
 - **Organic**
 - **Semidetached**
 - **Embedded**

COCOMO

- **Organic:** In this type of project is well understood with a small development team having good experience in a similar project.
- These projects are simple, and the project size is in the range (of 2KLOC –50 KLOC).

COCOMO

- **2) Semidetached:** In this type of project, the development team is a mixture of experienced and inexperienced team members.
- These projects are used for creating utility programs in the range of (50KLOC –300 KLOC)

COCOMO

- **Embedded:** In this type of project, the software to be developed is **strongly connected** to the **complex hardware**.
- **A development team** needs to know **embedded system-based software** of this type.
- These are **complex projects** with **restricted regulations** and a **size >300 KLOC**.

COCOMO - Basic

- The **basic COCOMO model** gives an **approximate estimate** of the project parameters.

$$\text{Effort} = a_1 \times (\text{KLOC})^{a_2} \text{ PM}$$

$$\text{Tdev} = b_1 \times (\text{Effort})^{b_2} \text{ Months}$$

- **KLOC:** Kilo Lines of Code as an estimated size of the project
- **a1, a2, b1, b2:** Constant values for each category of the software products
- **Tdev:** estimated time to develop the software, expressed in months
- **PMs:** Total effort required to develop the software product, expressed in person-months.

COCOMO - Basic

Project	a1	a2	b1	b2
Organic	2.4	1.05	2.5	0.38
Semi Detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

COCOMO - Basic

Estimation of development effort: The formulas for estimating the effort for the **three classes of software products**, based on the code size are shown below:

Organic	:	$\text{Effort} = 2.4(KLOC)^{1.05}$	PM
Semi-detached	:	$\text{Effort} = 3.0(KLOC)^{1.12}$	PM
Embedded	:	$\text{Effort} = 3.6(KLOC)^{1.20}$	PM

Estimation of development time:

Organic	:	$T_{dev} = 2.5(\text{Effort})^{0.38}$	Months
Semi-detached	:	$T_{dev} = 2.5(\text{Effort})^{0.35}$	Months
Embedded	:	$T_{dev} = 2.5(\text{Effort})^{0.32}$	Months

Example

- Example: Assume that the size of an **organic type** of software product has been estimated to be **32,000** lines of source code. Assume that the average salary of software engineers is **Rs. 15,000/-** per month. **Determine the effort** required to **develop the software product** and the **nominal development time**.
- Solution:
- From the **basic COCOMO** estimation formula for organic software:
- **Effort = $2.4 \times (32)^{1.05} = 91$ PM**
- **Nominal development time = $2.5 \times (91)^{0.38} = 14$ months**
- **Cost required to develop the product = $14 \times 15,000$**
= Rs. 210,000/-

Unit 5 – Maintenance

COCOMO – Intermediate and Complete

COCOMO - Intermediate

- The **basic COCOMO model determines efforts using LOC** which is only a function of the size/ number of lines of code and some.
- But in practice, **only LOC cannot be used** to determine any **system's efforts and schedule**.
- The other factors such as **experience, capability, and reliability** must be considered should be used in **estimating efforts**. This is included in **COCOMO – Intermediate Model**.
- These **factors** are known as **multipliers** or **cost drivers**. There are **15 such drivers** used in cost estimation.

COCOMO - Intermediate

- **Attributes in COCOMO – Intermediate are:**
- **Product**
- **Personnel**
- **Computer**
- **Development Environment**

COCOMO - Intermediate

- The **cost drivers** are **classified** based on the attributes of the following items.
- **Product:** The complexity, size of the database, and reliability of the product are considered in this attribute.
- **Computer:** The configuration of the computer such as execution speed, memory storage, run time etc. is considered in this attribute.

COCOMO - Intermediate

- **Personnel:** The **programming and product experience** of the software developer, **knowledge**, and **analytical ability** is considered in this attribute.
- **Development Environment:** The **facilities such as tools and methods available for software development** are considered in this attribute. One of the important features that are required in this attribute is **CASE tools** for **the readymade availability** of the software components.

COCOMO - Intermediate

□ $E = (a(KLOC^b) * EAF$

EAF: Effort Adjustment Factor obtained from the **15 different parameters** of the project as mentioned above. The value of a and b are as follows:

Software Projects	a	b
Organic	3.2	1.05
Semi Detached	3.0	1.12
Embedded	2.8	1.20

COCOMO - Complete

- In the **basic and intermediate COCOMO model**, the **software product** is considered a **homogeneous entity**.
- Most of the **large systems** consist of many **smaller sub-systems**. These subsystems may be **organic type, semidetached, or embedded**.
- The **complexity** of these **subsystems** may be **different**.
- The **complete COCOMO model** considers these characteristics and estimates the **efforts and development time** as the **sum of the estimates** for the individual subsystems.

COCOMO - Complete

- The cost driver's impact on each step of the software engineering process is considered.
- The COCOMO is applied in each phase to estimate effort and then the efforts are added.
- The Six phases of detailed COCOMO are:
 - Planning and requirements
 - System design
 - Detailed design
 - Module code and test
 - Integration and test
 - Cost Constructive model

Example of Complete COCOMO

- Consider a distributed Management Information System (MIS) product for an organization.
- The organization is spread across the country with offices in several places.
- Each office can have sub-components such as a database, graphical user interface (GUI), and communication part.

Example of Complete COCOMO

- Here the **communication** is of the type of embedded software.
- The **database part** is **semi-detached software** and **GUI** is of the type **organic software type**.
- The **overall cost of the system** is estimated as the **sum of the three subcomponents**.

COCOMO - Advantages

- **COCOMO** is transparent and useful to understand the impact of different factors that affect project costs.
- **COCOMO** includes ideas about the projects in the past.
- The drivers are helpful to understand the impact of hardware, persons, software tools, and development atmosphere on software development and cost.
- **COCOMO** is easy to use for cost estimation

COCOMO - Drawbacks

- COCOMO depends on the historical data and in some cases, it may not be available.
- Client's skills, knowledge, and cooperation are ignored in COCOMO.
- If the software project type is not classified correctly then COCOMO delivers incorrect results.
- Time estimation may be incorrect using COCOMO as it depends on a few factors, and this affects the project.

Unit 5 – Maintenance

Analytical Estimation – Halstead Techniques

Analytical estimation techniques

- In this technique, the given task is divided into basic components.
- If there is time calculation available from some other source, then the same is applied to all the components.
- If such time is not available, then the estimation of work is done using the experience of the work.
- This is done using Halstead's software science

Halstead's Software Science: Metrics

- Length and Vocabulary
- Program Volume
- Potential Minimum Volume
- Program Level
- Effort
- Time

Halstead's Software Science

- Halstead's software science is an analytical technique used to measure the size of the software, development effort, and software products.
- Halstead method uses a few program parameters. For a given program, let:
 - η_1 be the number of unique operators used in the program,
 - η_2 be the number of unique operands used in the program,
 - N_1 is the total number of operators used in the program,
 - N_2 is the total number of operands used in the program.
 - η_1 and η_2 are pronounced as eta1 and eta2

Length and Program Vocabulary

- The length of a program is considered as the sum of all operators and operands in the program.
- $\text{Length} = N = N_1 + N_2$
- The unique operators and operands used in the program are given as program vocabulary and it is given as follows:
 - $\eta = \eta_1 + \eta_2$

Program Volume

- When the programming style is different, then using only operators and operands is not enough.
- The project size can be estimated using program volume.
- Program volume is measured as the minimum number of bits needed to encode a program. It is given as:
- $V = N \log_2 \eta$
- Here η refers to the different identifiers and N is the length of the program.

Potential Minimum Volume

- The volume can be defined as potential minimum volume V^* .
- It refers to that part of the program which is clear and relates to the actual problem.
- V^* is given as:
- $V^* = (2 + \eta_2) \log_2 (2 + \eta_2)$

Program level

- The **program level** is used to measure the **level of abstraction** provided by the programming language.
- There can be **different levels** in the **program**, and it is given as:
- $L = V^*/V$

Effort

- **Effort** unit is used to **read and understand** the **program**.
- It refers to the **number of mental efforts** required to convert an **algorithm** to a **computer program**.
- Effort is given as:
- **Effort $E = V/L$,**
- $E = V^2/V^*$ (since $L = V^*/V$)

Time and Length

- **Time:** The time required by the programmer is given as $T = E/S$
- where **S** is the speed of a person's brain's processing rate. It is determined by **psychological reasoning**.
- Halstead's method recommends the use of unique operators and operands to determine the length of the program. The length is given as:
- Length Estimation: $N = \eta_1 \log_2 \eta_1 + \eta_2 \log_2 \eta_2$

Unit 5 – Maintenance

Halstead's Software Metric Example - 1

Halstead's Software Science: Example

- • Let us consider the following C program:
- **main ()**
- **{**
- **int x, y, z, avg;**
- **scanf("%d %d %d", &x, &y, &z);**
- **avg = (x+y+z)/3**
- **printf("avg = %d", avg);**
- **}**

Halstead's Software Science: Example

- • The unique operators(η_1) are:
- **main, (), {}, int, scanf, &, ",", ":", "=", +, /, printf**
- • The unique operands(η_2) are:
- **x, y, z, &x, &y, &z, x+y+z, avg, 3, "%d %d %d",**
 "avg = %d"
- • Therefore,
- **$\eta_1 = 12, \eta_2 = 11$**

Halstead's Software Science: Example

- $\eta_1 = 12, \eta_2 = 11$
- Estimated Length = $(12 \cdot \log_2 12 + 11 \cdot \log_2 11)$,
- (since $N = \eta_1 \log_2 \eta_1 + \eta_2 \log_2 \eta_2$)
- $= (12 \cdot 3.58 + 11 \cdot 3.45)$
- $= (43 + 38) = 81$
- Volume = Length $\cdot \log_2 (23)$,
- (since Volume $V = N \log_2 \eta$)
- $= 81 \cdot 4.52$
- $= 366$

Unit 5 – Maintenance

Halstead's Software Metric Example - 2

Halstead's Software Science Example

```
void area(int r)
{
    float a;
    a = 3.14 * r * r;
    printf("area is %f", a);
}
```

$\eta_1 = 10, \eta_2 = 4$

$N_1 = 12, N_2 = 8$

Operator	Operator	Occurrence
1	int	1
2	()	1
3	{ }	1
4	;	3
5	=	1
6	*	2
7	printf	1
8	,	1
9	float	1
10	void	1
	Total	12

Operand	Occurrence
r	3
a	3
3.14	1
Area is %f	1
Total	8

Length and Volume

- Count of Total occurrences of Unique operators $N_1 = 12$
- Count of Total occurrences of unique operands $N_2 = 8$
- Count of Unique operators $\eta_1 = 10$
- Count of unique operands $\eta_2 = 4$
- Estimated Length is given as $N = \eta_1 \log_2 \eta_1 + \eta_2 \log_2 \eta_2$
- $= 10 * 3.32 + 4 * 2$
- $= 41.2$
- Volume = Length* $\log_2 \eta$
- $\eta = 10 + 4 = 14$
- $V = 41.2 * 3.80$
- $V = 156.56$

Potential Minimum Volume

- $V^* = (2 + \eta_2) \log_2(2 + \eta_2)$
- $V^* = (6)^* \log_2(6) = 14.58$
- $L = V^*/V = 14.58/156.56 = 0.0931$
- Effort $E = V/L = 156.56/0.056 = 2795.71$
- Time = $E/S = 2795.71/18 = 155$ Seconds
- *(18 is assumed for mental discrimination)*